

Übungsstunde Woche 4

Fynn Krebs

7.10.2026



50:50

Pub

Tel

Frage 1 ■ 50 €

Was ist keine Schleifenart in C++?

A: While-Schleife

B: Repeat-Until-Schleife

C: For-Schleife

D: Do-While-Schleife

15 1 000 000 €

14 500 000 €

13 125 000 €

12 64 000 €

11 32 000 €

10 16 000 €

9 8 000 €

8 4 000 €

7 2 000 €

6 1 000 €

5 500 €

4 300 €

3 200 €

2 100 €

1 50 €

50:50

Pub

Tel

Frage 1 ■ 50 €

Was ist keine Schleifenart in C++?

A: While-Schleife

B: Repeat-Until-Schleife

C: For-Schleife

D: Do-While-Schleife

15 1 000 000 €

14 500 000 €

13 125 000 €

12 64 000 €

11 32 000 €

10 16 000 €

9 8 000 €

8 4 000 €

7 2 000 €

6 1 000 €

5 500 €

4 300 €

3 200 €

2 100 €

1 50 €

50:50

Pub

Tel

Frage 2 ■ 100 €

Was beschreibt den Typ `void` am besten?

A: Funktion hat keine Parameter

B: Funktion hat weder Parameter, noch Rückgabewert

C: Funktion kann nicht aufgerufen werden

D: Funktion hat keinen Rückgabewert

15 1 000 000 €

14 500 000 €

13 125 000 €

12 64 000 €

11 32 000 €

10 16 000 €

9 8 000 €

8 4 000 €

7 2 000 €

6 1 000 €

5 500 €

4 300 €

3 200 €

2 100 €

1 50 €

50:50

Pub

Tel

Frage 2 ■ 100 €

Was beschreibt den Typ `void` am besten?

A: Funktion hat keine Parameter

B: Funktion hat weder Parameter, noch Rückgabewert

C: Funktion kann nicht aufgerufen werden

D: Funktion hat keinen Rückgabewert

15 1 000 000 €

14 500 000 €

13 125 000 €

12 64 000 €

11 32 000 €

10 16 000 €

9 8 000 €

8 4 000 €

7 2 000 €

6 1 000 €

5 500 €

4 300 €

3 200 €

2 100 €

1 50 €

50:50

Pub

Tel

Frage 3 ■ 200 €

Lässt sich leicht prüfen, ob in einem Programm eine unendliche Schleife existiert?

A: Ja, das geht immer einfach

B: Nein, im Allgemeinen nicht

C: Nur bei for-Schleifen

D: Nur bei `while(true)`

15 1 000 000 €

14 500 000 €

13 125 000 €

12 64 000 €

11 32 000 €

10 16 000 €

9 8 000 €

8 4 000 €

7 2 000 €

6 1 000 €

5 500 €

4 300 €

3 200 €

2 100 €

1 50 €

50:50

Pub

Tel

Frage 3 ■ 200 €

Lässt sich leicht prüfen, ob in einem Programm eine unendliche Schleife existiert?

A: Ja, das geht immer einfach

B: Nein, im Allgemeinen nicht

C: Nur bei for-Schleifen

D: Nur bei `while(true)`

15 1 000 000 €

14 500 000 €

13 125 000 €

12 64 000 €

11 32 000 €

10 16 000 €

9 8 000 €

8 4 000 €

7 2 000 €

6 1 000 €

5 500 €

4 300 €

3 200 €

2 100 €

1 50 €

50:50

Pub

Tel

Frage 4 ■ 300 €

Alte Prüfungsfrage: Welche Aussage beschreibt die Ausgabe des folgenden Ausdrucks am besten?

```
std::cout << (true && (true && false || true));
```

A: 0/false**B:** 1/true**C:** Kompilierfehler**D:** Laufzeitfehler

15 1 000 000 €

14 500 000 €

13 125 000 €

12 64 000 €

11 32 000 €

10 16 000 €

9 8 000 €

8 4 000 €

7 2 000 €

6 1 000 €

5 500 €

4 300 €

3 200 €

2 100 €

1 50 €

50:50

Pub

Tel

Frage 4 ■ 300 €

Alte Prüfungsfrage: Welche Aussage beschreibt die Ausgabe des folgenden Ausdrucks am besten?

```
std::cout << (true && (true && false || true));
```

A: 0/false

B: 1/true

C: Kompilierfehler

D: Laufzeitfehler

15 1 000 000 €

14 500 000 €

13 125 000 €

12 64 000 €

11 32 000 €

10 16 000 €

9 8 000 €

8 4 000 €

7 2 000 €

6 1 000 €

5 500 €

4 300 €

3 200 €

2 100 €

1 50 €

XOR (Aufgabe 1.4)

- Wie programmiert man a oder b aber nicht beide?

XOR (Aufgabe 1.4)

- Wie programmiert man a oder b aber nicht beide?
- `(a || b) && !(a && b)`

XOR (Aufgabe 1.4)

- Wie programmiert man a oder b aber nicht beide?
- `(a || b) && !(a && b)`
- `a != b`

Serie letzter Woche

- **Natural Language** -> **C++**: Achtung, Differenz von a und b ist $|a - b|$, nicht $a - b$.
- **Fibonacci**: Meiner Meinung tricky Aufgabe, aber sehr gut gelöst von euch!
- π : Schleifen mit Summen und Produkten. Dennoch super gelöst! Schauen wir heute nochmals genauer an.

Fragen?

Ziel von Heute

- Verständnis der verschiedenen Schleifenarten
- Berechnung von Summen und Produkten

While Schleifen

- Wiederholt Code, **solange** die Bedingung wahr ist
- Die Bedingung wird **vor** jedem Durchlauf geprüft

```
int i = 0;  
    condition  
    ↓  
while (i < 10) {  
    std::cout << i << std::endl;  
    i++;  
}
```

While Schleifen

- Wiederholt Code, **solange** die Bedingung wahr ist
- Die Bedingung wird **vor** jedem Durchlauf geprüft

```
int i = 0;  
    condition  
    ↓  
while (i < 10) {  
    std::cout << i << std::endl;  
    i++;  
}
```

1. Bedingung überprüfen. Wenn Falsch, Schleife verlassen.
2. Schleifenrumpf ausführen (inkl. `i++` o.ä.)
3. Zurück zu Schritt 1

Do-While Schleifen

- Wie while, aber die Bedingung wird **nach** dem Durchlauf geprüft
- Der Rumpf wird daher **mindestens einmal** ausgeführt

```
int i = 0;  
do {  
    std::cout << i << std::endl;  
    i++;  
} while (i < 10);  
      ↑  
    condition
```

Do-While Schleifen

- Wie while, aber die Bedingung wird **nach** dem Durchlauf geprüft
- Der Rumpf wird daher **mindestens einmal** ausgeführt

```
int i = 0;  
do {  
    std::cout << i << std::endl;  
    i++;  
} while (i < 10);
```


condition

1. Schleifenrumpf ausführen
2. Bedingung überprüfen
3. Wenn Wahr: zurück zu Schritt 1
4. Wenn Falsch: Schleife verlassen

Schleifen im Vergleich

	Vorteile	Nachteile
<code>for</code>	▪ Init, Bedingung und Update in einer Zeile ▪ Anzahl der Durchläufe sofort ersichtlich	▪ Unflexibel bei unbekannter Anzahl Durchläufe ▪ Off-by-one-Fehler bei der Bedingung
<code>while</code>	▪ Sehr flexibel, beliebige Bedingung ▪ Rumpf läuft ggf. gar nicht	▪ Endlosschleife, wenn das Update fehlt ▪ Init und Update im Code verstreut
<code>do-while</code>	▪ Rumpf läuft mindestens einmal	▪ Auch hier Endlosschleife möglich

Schleifen im Vergleich

	Vorteile	Nachteile	Anwendungen
<code>for</code>	▪ Init, Bedingung und Update in einer Zeile ▪ Anzahl der Durchläufe sofort ersichtlich	▪ Unflexibel bei unbekannter Anzahl Durchläufe ▪ Off-by-one-Fehler bei der Bedingung	▪ Zählschleifen
<code>while</code>	▪ Sehr flexibel, beliebige Bedingung ▪ Rumpf läuft ggf. gar nicht	▪ Endlosschleife, wenn das Update fehlt ▪ Init und Update im Code verstreut	
<code>do-while</code>	▪ Rumpf läuft mindestens einmal	▪ Auch hier Endlosschleife möglich	

Schleifen im Vergleich

	Vorteile	Nachteile	Anwendungen
<code>for</code>	▪ Init, Bedingung und Update in einer Zeile ▪ Anzahl der Durchläufe sofort ersichtlich	▪ Unflexibel bei unbekannter Anzahl Durchläufe ▪ Off-by-one-Fehler bei der Bedingung	▪ Zählschleifen
<code>while</code>	▪ Sehr flexibel, beliebige Bedingung ▪ Rumpf läuft ggf. gar nicht	▪ Endlosschleife, wenn das Update fehlt ▪ Init und Update im Code verstreut	▪ Lesen bis Dateiende ▪ Warten auf ein Ereignis
<code>do-while</code>	▪ Rumpf läuft mindestens einmal	▪ Auch hier Endlosschleife möglich	

Schleifen im Vergleich

	Vorteile	Nachteile	Anwendungen
<code>for</code>	▪ Init, Bedingung und Update in einer Zeile ▪ Anzahl der Durchläufe sofort ersichtlich	▪ Unflexibel bei unbekannter Anzahl Durchläufe ▪ Off-by-one-Fehler bei der Bedingung	▪ Zählschleifen
<code>while</code>	▪ Sehr flexibel, beliebige Bedingung ▪ Rumpf läuft ggf. gar nicht	▪ Endlosschleife, wenn das Update fehlt ▪ Init und Update im Code verstreut	▪ Lesen bis Dateiende ▪ Warten auf ein Ereignis
<code>do-while</code>	▪ Rumpf läuft mindestens einmal	▪ Auch hier Endlosschleife möglich	▪ Eingabe validieren ▪ Menüs, die mind. einmal erscheinen

Drei gleiche Schleifen?

- Machen die drei Schleifen das gleiche? Warum?
- Kann man alle drei Schleifen unterscheiden?

```
int n; std::cin >> n;  
int i;
```

```
// loop 1  
for (i = 1; i <= n; ++  
    i) {  
    std::cout << i << "\n";  
}
```

```
// loop 2  
i = 0;  
while (i < n) {  
    std::cout << ++i <<  
        "\n";  
}
```

```
// loop 3  
i = 1;  
do {  
    std::cout << i++ <<  
        "\n";  
} while (i <= n);
```

Drei gleiche Schleifen?

- Machen die drei Schleifen das gleiche? Warum?
- Kann man alle drei Schleifen unterscheiden?

```
int n; std::cin >> n;  
int i;
```

```
// loop 1  
for (i = 1; i <= n; ++  
    i) {  
    std::cout << i << "\n";  
}
```

```
// loop 2  
i = 0;  
while (i < n) {  
    std::cout << ++i <<  
        "\n";  
}
```

```
// loop 3  
i = 1;  
do {  
    std::cout << i++ <<  
        "\n";  
} while (i <= n);
```

- Schleife 3 gibt für $n = 0$ eine Ausgabe, die anderen nicht.
- Für n als maximaler Wert von `int` haben Schleife 1 und 3 einen Überlauf, Schleife 2 nicht.

Schleifen umbauen

Schreibe die folgenden Schleifen jeweils in eine äquivalente Schleife um:

Von einer `for`-Schleife zu einer `while`-Schleife:

```
for (int i = 0 ; i < n ; i++){  
    BODY  
}
```

Schleifen umbauen

Schreibe die folgenden Schleifen jeweils in eine äquivalente Schleife um:

Von einer `for`-Schleife zu einer `while`-Schleife:

```
for (int i = 0 ; i < n ; i++){  
    BODY  
}
```

Von einer `while`-Schleife zu einer `for`-Schleife:

```
while (condition) {  
    BODY  
}
```

Schleifen umbauen

Schreibe die folgenden Schleifen jeweils in eine äquivalente Schleife um:

Von einer **for**-Schleife zu einer **while**-Schleife:

```
for (int i = 0 ; i < n ; i++){  
    BODY  
}
```

Von einer **while**-Schleife zu einer **for**-Schleife:

```
while (condition) {  
    BODY  
}
```

Von einer **do-while**-Schleife zu einer **for**-Schleife:

```
do {  
    BODY  
} while (condition);
```

Fakultäten

Wir wollen $n! = \prod_{i=1}^n i$ berechnen.

- Was für Variablen brauchen wir?

Fakultäten

Wir wollen $n! = \prod_{i=1}^n i$ berechnen.

- Was für Variablen brauchen wir?
- Wie initialisieren wir diese Variablen? Wieso?

Wir wollen $n! = \prod_{i=1}^n i$ berechnen.

- Was für Variablen brauchen wir?
- Wie initialisieren wir diese Variablen? Wieso?
- Wie schreiben wir die Schleife? Welche Art von Schleife ist am besten geeignet?

Funktionen

- Fassen Code zu einem benannten, wiederverwendbaren Block zusammen
- Nehmen Parameter entgegen und geben einen Wert zurück

```
int add(int a, int b) {  
    return a + b;  
}
```

Funktionen

- Fassen Code zu einem benannten, wiederverwendbaren Block zusammen
- Nehmen Parameter entgegen und geben einen Wert zurück

return type

name

parameters

The diagram shows the function signature `int add(int a, int b) {` with red arrows pointing to its components: `int` is pointed to by the label **return type**, `add` is pointed to by **name**, and `(int a, int b)` is pointed to by **parameters**. The code itself is color-coded: `int` is blue, `add` is black, and `(int a, int b)` is blue.

```
int add(int a, int b) {  
    return a + b;  
}
```


Funktionen

- Fassen Code zu einem benannten, wiederverwendbaren Block zusammen
- Nehmen Parameter entgegen und geben einen Wert zurück

return type

name **parameters**

```
int add(int a, int b) {  
    return a + b;  
}
```

1. Aufruf: Die Argumente werden an die Parameter übergeben
2. Funktionsrumpf ausführen
3. `return` beendet Funktion und gibt Wert zurück
4. Das Programm läuft an Aufrufstelle weiter

Taylorreihen

- Benannt nach ~~Taylor Swift~~ Brook Taylor (1685-1731)
- Motivation: Polynome sind "einfache" Funktionen.
- Taylorreihen erlauben es, komplizierte Funktionen als Polynome zu approximieren (Siehe Analysis I)¹.
- Beispiel: $\sin(x)$ kann als unendliche Reihe dargestellt werden:

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

¹Computer verwenden typischerweise alternative Algorithmen, da Divisionen langsam sind.

Taylorreihen

$$\sin(x) = \sum_{n=0}^{\infty} \underbrace{\frac{(-1)^n}{(2n+1)!} x^{2n+1}}_{:=a_n} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Wir wollen nun die Taylorreihe von $\sin(x)$ verwenden, um $\sin(x)$ auf eine Genauigkeit von 0.000001 zu approximieren.

Überlegungen:

1. Angenommen, wir kennen den Term a_n , wie bestimmen wir a_{n+1} ?
2. Was für eine Schleife sollten wir verwenden, um die Summe zu berechnen? Was ist die Condition?

Taylorreihen

$$\sin(x) = \sum_{n=0}^{\infty} \underbrace{\frac{(-1)^n}{(2n+1)!} x^{2n+1}}_{:=a_n} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Wir wollen nun die Taylorreihe von $\sin(x)$ verwenden, um $\sin(x)$ auf eine Genauigkeit von 0.000001 zu approximieren.

<https://expert.ethz.ch> 

Für die Schnellen: Schafft ihr es, die Fakultäten und Potenzen nicht immer von vorne zu berechnen?

Scopes

- Definiert, wo Variablen gültig sind
- Variablen sind nur innerhalb des Scopes gültig.
- Merkregel: Immer wenn Klammern geschlossen werden, geht alles was darin definiert wurde verloren.

```
1 int main() {  
2     int x = 5;  
3     {  
4         int x = 3;  
5         std::cout << x << std::endl; // Ausgabe: ???  
6     }  
7     std::cout << x << std::endl; // Ausgabe: ???  
8     return 0;  
9 }
```

Scopes

Was ist der Scope der Variable `sum`, `i`, `a` im folgenden Code?

```
1  int sum = 0;
2  for (int i = 0 ; i < 5 ; ++i){
3      int a;
4      std::cin >> a;
5      sum += a;
6  }
```

Jump Statements

Was ist "unschön" an diesem Code?

```
for (int i = 0 ; i < N ; i++){  
    if (i%2 == 0){  
        if (i%3 == 1){  
            if (i%5 == 2){  
                std::cout << i << std::endl;  
            }  
        }  
    }  
}
```

Jump Statements

Besser?

```
for (int i = 0 ; i < N ; i++){  
    if (i%2 != 0){  
        // Nichts machen  
    }  
    else if (i%3 != 1){  
    }  
    else if (i%5 != 2){  
    }  
    else {  
        std::cout << i << std::endl;  
    }  
}
```


Jump Statements

Mit *Guard Clause*

```
for (int i = 0 ; i < N ; i++){  
    if (i%2 != 0) continue;  
    if (i%3 != 1) continue;  
    if (i%5 != 2) continue;  
  
    std::cout << i << std::endl;  
}
```

- Alternativ kann man Bedingungen auch in ein if statement packen, wird aber schnell unübersichtlich.

Tipps zu den Übungen

Loop mix-up

Tipps zu den Übungen

Loop mix-up

- Schaut nochmals die Tabelle zu Vor- und Nachteilen der Loops an.

Tipps zu den Übungen

Loop mix-up

- Schaut nochmals die Tabelle zu Vor- und Nachteilen der Loops an.
- PythonTutor  kann helfen.

Tipps zu den Übungen

Loop mix-up

- Schaut nochmals die Tabelle zu Vor- und Nachteilen der Loops an.
- PythonTutor [↗](#) kann helfen.

Loop Analysis

Tipps zu den Übungen

Loop mix-up

- Schaut nochmals die Tabelle zu Vor- und Nachteilen der Loops an.
- PythonTutor [↗](#) kann helfen.

Loop Analysis

- Auch hier kann PythonTutor [↗](#) helfen.